

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Total Marks:50

Duration : 1 Hour

Annexure A

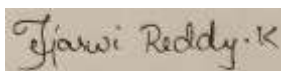
Analytical Problem Solving

Code of Conduct:

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

SOLUTION –

- Both jugs are initially empty.
- There are no measuring marks on either jug.
- Water supply from the pump is unlimited.
- Water may be discarded onto the ground.
- Water can be transferred freely between the two jugs.

A state is represented as **(4G, 3G)**, where:

- **4G** = Amount of water in the 4-gallon jug.
- **3G** = Amount of water in the 3-gallon jug.

Initial State: (0, 0)

Goal State: (2, 0)

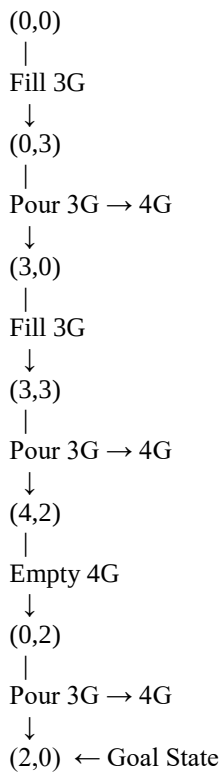
Algorithm

1. Fill the 3-gallon jug.
2. Pour the water into the 4-gallon jug.
3. Fill the 3-gallon jug again.
4. Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug becomes full.
5. Empty the 4-gallon jug.
6. Pour the remaining water from the 3-gallon jug into the 4-gallon jug.
7. The 4-gallon jug now contains exactly 2 gallons of water.

Step-by-Step Solution

Step	Operation	State (4G, 3G)
Initial	Both jugs are empty	(0, 0)
1	Fill the 3-gallon jug	(0, 3)
2	Pour 3G into 4G	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour from 3G to 4G until 4G is full	(4, 2)
5	Empty the 4-gallon jug	(0, 2)
6	Pour the remaining 2 gallons from 3G into 4G	(2, 0)

State Space Diagram



2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?

SOLUTION –

- i. What are the Percepts for this Agent?

Percepts are the information received by the rover through its sensors.

The Mars Rover receives the following percepts:

- Images from navigation and scientific cameras.
- Surface and terrain information.
- Rock and soil composition data.
- Temperature, atmospheric pressure, and weather conditions.
- Distance to obstacles.
- Position and orientation (GPS-like localization and inertial sensors).
- Battery and power level.
- Wheel traction and movement status.
- Radiation and environmental measurements.

ii. Characterize the Operating Environment

The operating environment of a Mars Rover can be characterized as follows:

Environment Property	Description
Observability	Partially Observable – The rover cannot observe the entire environment at once.
Determinism	Stochastic – Outcomes are uncertain due to terrain, weather, and wheel slippage.
Episodic/Sequential	Sequential – Current actions affect future decisions and missions.
Static/Dynamic	Dynamic – Environmental conditions such as dust storms may change over time.
Discrete/Continuous	Continuous – Position, movement, and sensor values change continuously.
Single/Multi-Agent	Single Agent – The rover performs its mission independently.
Known/Unknown	Partially Known – Maps and terrain information are incomplete before exploration.

iii. What are the Actions the Agent Can Take?

The Mars Rover can perform the following actions:

- Move forward, backward, left, or right.
- Avoid obstacles.
- Capture images and videos.
- Drill rocks and collect soil samples.
- Analyze samples using onboard scientific instruments.
- Measure atmospheric and environmental conditions.
- Store scientific data.
- Transmit collected data to Earth.
- Recharge batteries using solar panels (when sunlight is available).
- Stop or enter safe mode during hazardous conditions.

iv. How Can One Evaluate the Performance of the Agent?

The performance of the Mars Rover can be evaluated using the following criteria:

- Accuracy of scientific observations and sample analysis.
- Successful completion of exploration objectives.
- Number and quality of samples collected.
- Safe navigation without collisions.
- Efficient use of battery power.
- Reliable communication with Earth.
- Distance covered during exploration.
- Ability to avoid hazards and recover from unexpected situations.
- Overall mission success and operational lifetime.

v. What Sort of Agent Architecture is Most Suitable? Why?

The most suitable architecture for a Mars Rover is a **Utility-Based Learning Agent**.

Reason:

- It continuously learns from sensor data and previous experiences.
- It selects actions that maximize mission success while minimizing risks.
- It balances multiple objectives such as safety, energy conservation, and scientific exploration.
- It adapts to unknown environments and changing terrain.
- It can make intelligent autonomous decisions when communication with Earth is delayed.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

SOLUTION

1. Initial State

- The chessboard is empty.
- No queens are placed on the board.

2. State Space

- A state represents the arrangement of queens placed on the chessboard.
- Each new state is generated by placing one queen in the next column while ensuring it does not attack any previously placed queen.

3. Actions (Operators)

The possible actions are:

- Place a queen in any safe row of the current column.
- Move to the next column after successfully placing a queen.
- If no safe position exists, backtrack to the previous column and try another position.

4. Goal State

A goal state is reached when:

- All **8 queens are placed** on the board.

- No two queens attack each other.

5. Path Cost

- Every queen placement has an equal cost.
- The objective is to find any valid solution rather than minimizing cost.

Search Space

The search space consists of all possible ways of placing eight queens.

- Total possible arrangements (without constraints): $8^8 = 16,777,216$
- If one queen is placed in each row and column: $8! = 40,320$
- Only **92** valid solutions exist.
- Considering symmetry, there are **12 unique** solutions.

AI search techniques efficiently eliminate invalid states, reducing the number of configurations that must be explored.

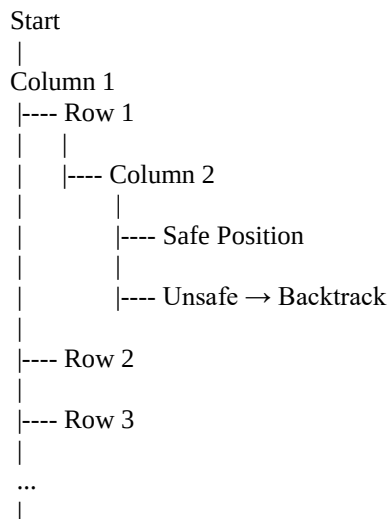
Search Strategy

The most common strategy used is **Backtracking Search (Depth-First Search)**.

Algorithm

1. Start with an empty chessboard.
2. Place a queen in the first column.
3. Move to the next column and place a queen in a safe position.
4. Continue placing queens column by column.
5. If no safe position exists, backtrack to the previous column and try another row.
6. Repeat until all eight queens are placed successfully.

State Space Tree (Concept)



Goal State (8 Safe Queens)

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

SOLUTION –

Type of Problem-Solving Agent

The **Goal-Based Agent** is the most suitable problem-solving agent for this application.

Reason

A Goal-Based Agent selects actions that help achieve a specific goal.

Goal: Book the most suitable cab to transport the customer from the pickup location to the destination.

The agent considers:

- Pickup location
- Destination
- Available cab types
- Fare estimate
- Driver availability
- Estimated arrival time (ETA)
- Customer preferences

It evaluates these factors and chooses the best option to achieve the user's goal.

Problem Formulation

Initial State

- Customer opens the OLA application.
- Pickup and destination are not entered.
- No cab is selected.

State Space

The state space includes all possible combinations of:

- Pickup locations

- Destination locations
- Available drivers
- Cab categories (Mini, Micro, Sedan, Prime, Shared, etc.)
- Fare estimates
- Estimated arrival times

Actions

The agent can perform the following actions:

1. Accept pickup location.
2. Accept destination location.
3. Search for nearby drivers.
4. Display available cab types.
5. Estimate fare and travel time.
6. Allow customer to select a cab.
7. Book the selected cab.
8. Assign a nearby driver.
9. Track the ride.
10. Complete payment after the trip.

Goal State

- A suitable cab is successfully booked.
- The customer reaches the destination safely.

Pseudocode

START

Input Pickup_Location

Input Destination

Search Available_Cabs

IF Available_Cabs = TRUE THEN

Display Cab Types

Display Fare and ETA

Customer Selects Cab_Type

Assign Nearest Driver

IF Driver Accepts Ride THEN

Confirm Booking

Track Ride

Reach Destination

Make Payment

Display "Trip Completed Successfully"

ELSE

Search Another Driver

ENDIF


```
ELSE
    Display "No Cabs Available"
ENDIF

STOP
```

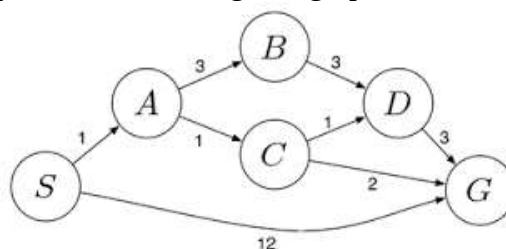
Flow of the Agent

```
Start
|
Enter Pickup Location
|
Enter Destination
|
Search Available Cabs
|
Display Cab Types
|
Select Cab
|
Assign Driver
|
Booking Confirmed
|
Track Ride
|
Reach Destination
|
Payment
|
End
```

Advantages of Using a Goal-Based Agent

- Selects the most suitable cab according to the user's requirements.
- Minimizes travel time and waiting time.
- Provides fare estimation before booking.
- Automatically searches for another driver if the first driver declines.
- Improves customer convenience and efficiency.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse **G** using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph:



SOLUTION –

Algorithm Used

Uniform Cost Search (UCS)

Uniform Cost Search expands the node having the **lowest cumulative path cost** first. It guarantees the optimal (minimum-cost) solution when all edge costs are positive.

Graph Representation

From To Cost

S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

Step-by-Step UCS Execution

Step 1

Start at **S**

Frontier:

S (0)

Expand **S**

Possible paths:

- $S \rightarrow A = 1$
- $S \rightarrow G = 12$

Frontier:

A(1), G(12)

Step 2

Expand **A(1)**

New paths:

- $S \rightarrow A \rightarrow B = 4$
- $S \rightarrow A \rightarrow C = 2$

Frontier:

C(2), B(4), G(12)

Step 3

Expand **C(2)**

New paths:

- $S \rightarrow A \rightarrow C \rightarrow D = 3$
- $S \rightarrow A \rightarrow C \rightarrow G = 4$

Frontier:

D(3), B(4), G(4), G(12)

Step 4

Expand **D(3)**

New path:

- $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 6$

Frontier:

B(4), G(4), G(6), G(12)

Step 5

Expand **B(4)**

New path:

- $S \rightarrow A \rightarrow B \rightarrow D = 7$

Frontier:

G(4), G(6), D(7), G(12)

Step 6

Expand **G(4)**

Since **G** is the goal node, the search stops.

Least-Cost Path

$S \rightarrow A \rightarrow C \rightarrow G$

Cost Calculation

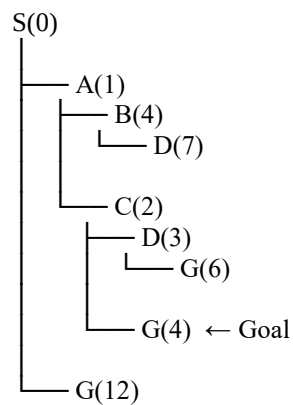
$S \rightarrow A = 1$

$A \rightarrow C = 1$

$C \rightarrow G = 2$

Total Cost = $1 + 1 + 2 = 4$

Search Tree



Pseudocode

START

Insert Start Node (S) into Priority Queue with Cost = 0

WHILE Priority Queue is not Empty

 Remove Node with Lowest Cost

 IF Node = Goal (G)

 Return Path and Total Cost

 Expand Current Node

 FOR each Neighbor

 New Cost = Current Cost + Edge Cost

 IF Neighbor not Visited OR New Cost is Smaller

 Insert Neighbor into Priority Queue

 ENDIF

ENDFOR

ENDWHILE

STOP

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem